

代码模型验证报告

Group 6 陈纳川 黄鑫 李书桓 朱奕宁

(一) 运行代码：可以顺利复现代码。

(二) 数据处理和样本选择

该组同学在最终的 Model_Clean_Price.ipynb 中，放弃了 DataClean_price_v2.py 中会导致泄露的目标编码方法，采用了新的留一法编码，并把该编码放在 6 折交叉验证的循环内部，这样可以避免编码过程使用验证折信息。对 StandardScaler 的使用也正确，只在训练折数据上 fit，再用它 transform 验证折，符合交叉验证的要求。

我在异常值处理步骤发现一个问题，该组同学在划分任何训练折和验证折之前，对完整数据集的目标值使用 IQR 方法移除异常值。团队在计算 Q1、Q3 和 IQR 时使用了全部样本。这样会让模型提前“看到”验证集的分布，模型在统一清洗后的数据上训练和评估。得到的 MAE 220k - 242k 不再代表模型对未来数据的真实表现。

(三) 改进建议

1. 模型现在的手动交互特征有效，但数量有限。自动特征工程脚本因内存不足而失败。可以先从 Lasso 的系数表选出前 20 个核心特征，再用 PolynomialFeatures (degree=2, interaction_only=True) 生成交互项，也可以使用 RFECV 和 Lasso 进行递归特征消除。这样可以得到更稳定的特征子集。
2. 模型需要把异常值处理放入交叉验证内部。团队应只使用训练折的目标值计算 Q1、Q3 和 IQR，再用这些边界处理训练折和验证折。这样可以避免验证折在预处理阶段暴露分布信息。也需要把当前的删除策略改为截断策略。可以在训练折计算边界之后，用 np.clip 限制训练折和验证折中的极端值。这样可以减少线性模型对极端值的敏感性，并保留全部样本的信息。

```
# 移除异常值 (使用 IQR 方法)
Q1 = y_train.quantile(0.25)
Q3 = y_train.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outlier_mask = (y_train >= lower_bound) & (y_train <= upper_bound)

X_tr = X_train[outlier_mask]
y_tr = y_train[outlier_mask]

print(f"异常值移除统计:")
print(f"  原始训练集样本数: {len(y_train):,}")
print(f"  移除异常值后样本数: {len(y_tr):,}")
print(f"  移除的样本数: {len(y_train) - len(y_tr):,}")
print(f"  移除比例: {(1 - len(y_tr)/len(y_train))*100:.2f}%")
print(f"\n  异常值移除完成")
```

3. 第七组在处理数据的过程中将大部分预处理步骤放在了另一个.py 文件中，这样的好处是可以大大压缩.ipynb 文件中的代码长度，同时增加代码的可复用性，但是这

样的问题就是每次运行无法单独运行一部分代码来快速排查错误，不利于其他人快速理解代码作用。对此我们给出的建议是可以考虑把一些大的函数放在.py 文件中，而处理的整体思路依然在.ipynb 文件中呈现。

4. 代码中用中位数填充缺失值是用全部的 test 数据的中位数来填充的，这在理论上会涉及一定的数据泄露，我们建议在用随机种子划分了训练集和测试集之后再进行缺失值的填充；同时，这里填充距“上次交易年数”的缺失值时应该用“据上次交易年数”的中位数，现在的代码用“交易距今年数”现实意义不明确。

```
# ===== 4) 日期处理函数 =====
def process_transaction_dates(df, current_year=2025):
    """ 将交易时间转化为"距今年数"和"距上次交易年数" """
    for col in ["交易时间", "上次交易"]:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce')
    if "交易时间" in df.columns:
        df["交易距今年数"] = current_year - df["交易时间"].dt.year
        # 使用中位数填充交易距今年数本身的缺失
        if df["交易距今年数"].isna().any():
            median_trade_age = df["交易距今年数"].median()
            df["交易距今年数"] = df["交易距今年数"].fillna(median_trade_age)
    if "交易时间" in df.columns and "上次交易" in df.columns:
        df["距上次交易年数"] = (df["交易时间"] - df["上次交易"]).dt.days / 365
        # 优先使用交易距今年数的中位数填充
        median_trade_age_fill = df["交易距今年数"].median() if "交易距今年数" in df.columns else 5 # 兜底值5年
        df["距上次交易年数"] = df["距上次交易年数"].fillna(median_trade_age_fill)
    return df
```

5. 梯户比例的处理有严重问题，该代码只能处理梯户数量是个位数的情况（代码直接将识别到的最后两个数作为梯、户的数量），但是原始数据中是包含两位数甚至三位数的数据的，对数据的挖掘不够细节，应该考虑把两位数和三位数转化为汉字的情况。

```
def parse_ladder_ratio(s):
    """ 解析梯户比例: '一梯三户' -> 1/3; '2梯4户' -> 0.5 """
    if pd.isna(s): return np.nan
    s = str(s)
    cn2num = {'一':1, '二':2, '两':2, '三':3, '四':4, '五':5, '六':6, '七':7, '八':8, '九':9, '十':10}
    digits = [cn2num.get(ch) for ch in s if ch in cn2num]
    if len(digits) >= 2 and digits[1] != 0: return digits[0] / digits[1]
    nums = re.findall(r"\d+", s)
    if len(nums) >= 2 and float(nums[1]) != 0: return float(nums[0]) / float(nums[1])
    return np.nan
```

- ✓ 一梯一百九十一户
- ✓ 一梯七户
- ✓ 一梯三十一户
- ✓ 一梯三十七户
- ✓ 一梯三十三户

6. 类似 3.，没有考虑到数据格式是“XX 房价”的情况，数据处理有误。

```
def parse_house_type(s):
    """ 解析户型: '3室2厅1厨2卫' -> (3, 2, 1, 2) """
    if pd.isna(s): return (np.nan, np.nan, np.nan, np.nan)
    s = str(s)
    get_first = lambda pat: float(re.findall(pat, s)[0]) if re.findall(pat, s) else np.nan
    return (get_first(r"(\d+)室"), get_first(r"(\d+)厅"), get_first(r"(\d+)厨"), get_first(r"(\d+)卫"))
```

☑ 2房间1卫

☑ 2房间2卫

- 该模型选择直接预测单价，但是套内面积本身也可能会影响单价，比如完全同样的地段，由于面积对住户的效用有边际递减效应，大面积的房屋单价可能更低，因此直接预测单价有所不妥。建议还是预测整体房价，将面积作为一个影响因素来处理。

```
train['Price/m2'] = train['Price'] / train['套内面积']
train, test = fill_missing_values(train, test)
```

- 这里删除无用列时选择删除了“板块”而保留“板块_comm”，但是事实上“板块”没有缺失值，而“板块_comm”包含缺失值，应该保留信息量更大的列，说明没有在一开始先分析每列的缺失值数量就直接开始处理了，建议在数据处理的最开始先输出各个类别的缺失值，同时比较内容重复的列中哪个列缺失值更少。

```
drop_cols_initial = [
    "开发商", "物业公司", "物业办公电话", "交易时间", "环线位置", "板块", # 通用
    "所在楼层", "梯户比例", "房屋户型", # 已解析
    "抵押信息", "上次交易" # 价格数据特有但通常无用
]
```

- 通过观察数据可以发现供热费的缺失值主要来源于该城市本身就不需要暖气供热，而房屋供热费对于需要供暖的城市来说依然是一个会影响房价的指标，因此不应该直接删除，建议将供热费的缺失值直接视为单独的一类来处理，而不是全部删除。

```
# 供热费在价格数据中缺失严重，直接删除
if '供热费' in train.columns: high_missing_cols.append('供热费')
high_missing_cols = list(set(high_missing_cols)) #去重
```

- 清洗脚本中的潜在目标泄露:

- 虽然 Notebook 里的 LOO 编码是正确的，但在清洗脚本 DataClean_price_v2.py 的 encode_geographic_features 函数中，板块_均价编码 是在清洗阶段基于全量训练集计算的。如果这个特征直接被放入模型训练（而不是像 Notebook 里那样被 LOO 特征替代），就会造成严重泄露。
- 建议：在清洗阶段不要生成与 Target 强相关的统计特征（如均价编码），只做清洗。将所有 Target Encoding 逻辑统一移到交叉验证循环内部进行。

- 互信息计算导致的内存溢出:

- mutual_info_regression 计算量非常大，且需要构建巨大的距离矩阵，特征多样

本大时必然爆内存。

- 建议：
 - 1) 降采样：只随机抽取 10,000 - 20,000 个样本计算互信息，通常足以反映特征重要性排序。
 - 2) 移除：既然已经有了 Lasso 和 Random Forest 的重要性排序，互信息并非必须。

12. 缺失值填充的数据一致性：

- 风险：测试集的填充利用了测试集自身的分布信息（test.groupby）。在实际业务或比赛中，测试集是“未来数据”，你看不到它的整体分布。
- 建议：应当保存训练集的 groupby 统计值（比如保存为一个字典或查找表），然后用这些训练集计算出的统计值去填充测试集。代码中 V2 版本已经尝试用 encoders 字典保存部分信息，建议将数值填充的统计量也保存进去。

13. 多项式特征的膨胀风险：

- 风险：代码中目前的特征数已经有 200+。如果对所有 Lasso 选出的特征做 2 次多项式，特征数量会爆炸（例如 50 个特征做 degree=2 会产生约 1275 个新特征），这会导致 Ridge/Lasso 训练变慢甚至过拟合。
- 建议：仅对 Lasso 系数绝对值最大的前 5-10 个核心特征（如面积、板块均价、室数等）做多项式交互，而不是前 20 个。

14. 在处理租金数据时，虽然使用了 HDBSCAN 进行地理聚类，但在处理未见过的测试集样本时，对于 cluster_id 为 -1 的噪声点，直接使用了“板块_价格编码”作为替代。这种做法可能不够稳健，建议对噪声点采用更精细的处理策略，比如基于空间最近邻的加权投票方法，确保训练集和测试集在聚类特征处理上的一致性。

```
[32]: mask = train_df['cluster_id'] == -1
      train_df.loc[mask, 'cluster_mean_price'] = train_df.loc[mask, '板块_价格编码']
```

```
[33]: mask = test_df['cluster_id'] == -1
      test_df.loc[mask, 'cluster_mean_price'] = test_df.loc[mask, '板块_价格编码']
```

15. 当前对于租金数据处理的特征工程主要沿用了房价预测的思路，但租房市场有其特殊性。建议针对租房场景开发更合适的特征，如考虑“交易距今月数”对租金的影响模式、不同区域的租金季节性变化特征，以及针对租房客户偏好的特定交互特征，这样能更好地捕捉租房市场的独特规律。
16. 代码中创建的“交易距今月数”特征虽然有用，但未能充分捕捉租房市场的季节性规律。租房市场通常有明显的学期周期、春节周期等季节性波动。建议进一步提取月份、季度等时间特征，并考虑不同城市/区域的季节性模式差异。特别是对于公寓、商务区住宅等特定类型的租房，其时间规律更为明显，细化时间特征能显著提升模型对租金波动规律的捕捉能力。

```
print("\n【第6步】解析交易时间...")
df['交易年份'] = df['交易时间'].apply(parse_transaction_date)
current_year = 2025
df['交易距今年数'] = current_year - df['交易年份']
```

17. 在租房场景中，某些费用（如供热费、燃气费）的缺失值可能具有特殊业务含义。例如，南方城市通常不需要供暖，因此供热费的缺失实际上反映了地域特性。当前代码用统一的中位数填充可能引入偏差，建议对这类特征进行缺失模式分析，将"因业务逻辑不需要而产生的缺失"单独作为一类处理，或用基于地域的分组统计值进行填充。

```
# 全局中位数填充
for col in existing_numeric_cols:
    if df[col].isnull().any():
        median_val = df[col].median()
        if pd.notna(median_val):
            df[col] = df[col].fillna(median_val)
        else:
            df[col] = df[col].fillna(0)
```